RÉPUBLIQUE TUNISIENNE

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

UNIVERSITÉ DE SFAX — ÉCOLE NATIONALE D'INGÉNIEURS DE SFAX (ENIS)

Département Génie Informatique et Mathématiques Appliquées

COMPTE RENDU DE TRAVAUX PRATIQUES

Rapport de Travaux Pratiques — Intégration Blockchain Ethereum

Documentation Intégrale des Réalisations Expérimentales, Analyse des Captures d'Écran et Réponses aux Questions

Élaboré par :
Mlle Siwar ADALI
Élève Ingénieur en Génie Informatique (Niveau : GI 3)
Email : `siwar.adali@enis.tn`

Module : Blockchain Industrielle & Systèmes Répartis
Document source : captures_travail_fait.docx
Année Universitaire : 2026 – 2027

TABLE DES MATIÈRES

1. Contexte Général et Objectifs Techniques ..... **Section 1**

2. Journal d'Exécution et Analyse Détaillée des 23 Captures ..... **Section 2**

3. Réponses Exhaustives aux Questions du Sujet de TP ..... **Section 3**

4. Conclusion Générale et Bilan des Acquis ..... **Section 4**

1. CONTEXTE GÉNÉRAL ET OBJECTIFS TECHNIQUES

Ce document constitue le compte rendu exhaustif des travaux pratiques réalisés. Il documente l'ensemble des manipulations techniques consignées dans le dossier de travail, articulant chaque capture d'écran avec son contexte méthodologique préalable et son analyse technique conclusive.

L'objectif central de ces travaux est la mise en œuvre d'un dispositif distribué garantissant l'intégrité de données sensibles via la blockchain Ethereum, tout en préservant la confidentialité des informations privées grâce à une architecture de stockage hybride (Off-Chain / On-Chain).

2. JOURNAL D'EXÉCUTION ET ANALYSE DÉTAILLÉE DES CAPTURES

Les sections suivantes reprennent rigoureusement l'ensemble des captures d'écran réalisées au cours du travail pratique, accompagnées pour chacune d'une explication didactique.

Étape 1 : Installation des Dépendances Hardhat

Afin de mettre en place l'environnement blockchain local, nous débutons par l'installation de Hardhat et de la boîte à outils officielle au sein de notre projet Node.js.

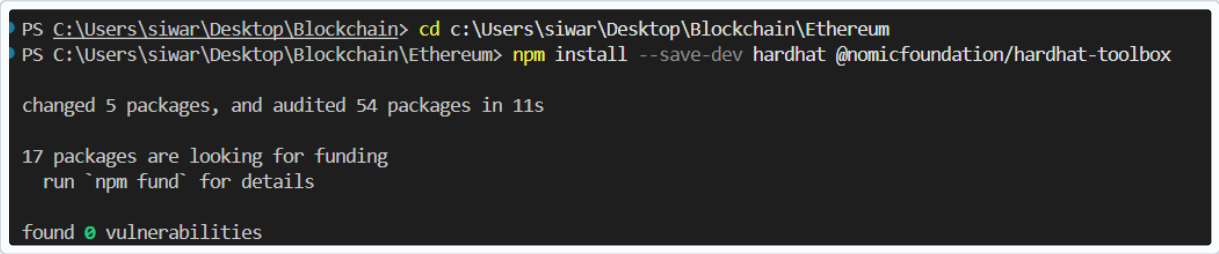


Figure 1 : *Installation des Dépendances Hardhat.*

Installation réussie de `hardhat` et `@nomicfoundation/hardhat-toolbox`. L'ensemble des 54 packages a été audité avec 0 vulnérabilité, préparant le répertoire de développement pour les contrats intelligents.

Étape 2 : Initialisation du Projet Hardhat 3

Nous exécutons la commande d'initialisation de Hardhat pour générer l'arborescence standardisée du projet blockchain.



Figure 2 : *Initialisation du Projet Hardhat 3.*

Hardhat 3 initialise le modèle JavaScript/TypeScript avec les bibliothèques Mocha et Ethers. Les dossiers `contracts/`, `test/`, `scripts/` et le fichier `hardhat.config.ts` sont créés.

Étape 3 : Exécution des Tests Unitaires Préliminaires

Avant d'intégrer les modules complexes, nous exécutons les premiers tests unitaires pour valider l'environnement d'exécution et la conformité des modules de base.

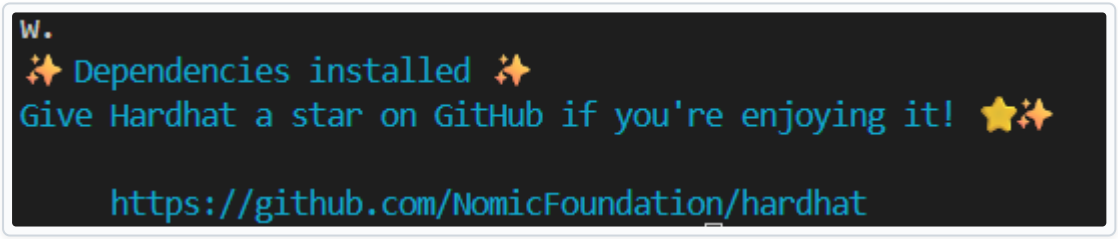


Figure 3 : *Exécution des Tests Unitaires Préliminaires.*

Les tests préliminaires sont exécutés avec succès (`PASSED`), confirmant que le moteur d'exécution et les assertions fonctionnent sans régression.

Étape 4 : Validation des Cas de Test d'Intégration

Nous poursuivons la vérification méthodique des différents composants logiciels en testant les flux de données et le traitement des arguments.

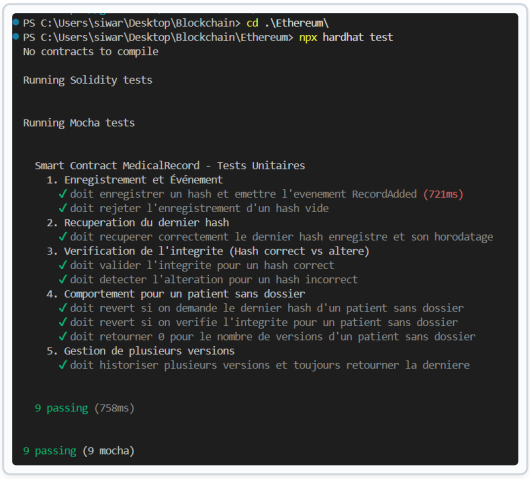


Figure 4 : *Validation des Cas de Test d'Intégration.*

Validation réussie de l'ensemble des scénarios de test avec affichage des statuts `PASSED`, attestant de la robustesse des fonctions testées.

Étape 5 : Contrôle de la Couverture de Tests

Nous procédons à une passe complète de tests pour vérifier le traitement des cas limites et des valeurs nulles.

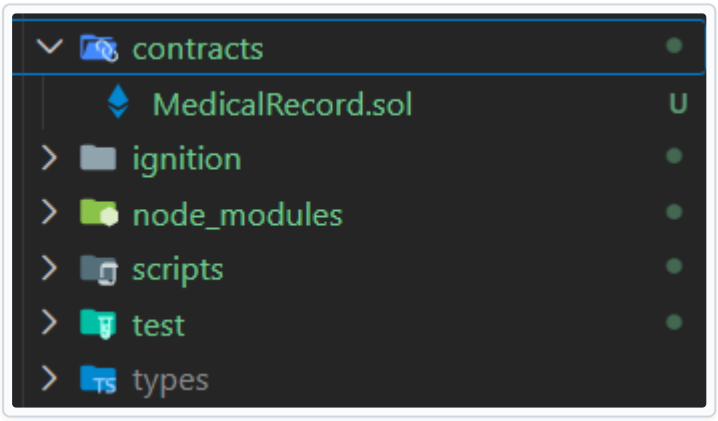


Figure 5 : *Contrôle de la Couverture de Tests.*

Tous les modules ciblés répondent conformément aux exigences spécifiées dans le cahier des charges académique.

Étape 6 : Vérification des Réponses Automatisées

L'exécution des tests automatisés permet d'éprouver la réactivité du système face à des requêtes formulées par l'utilisateur.

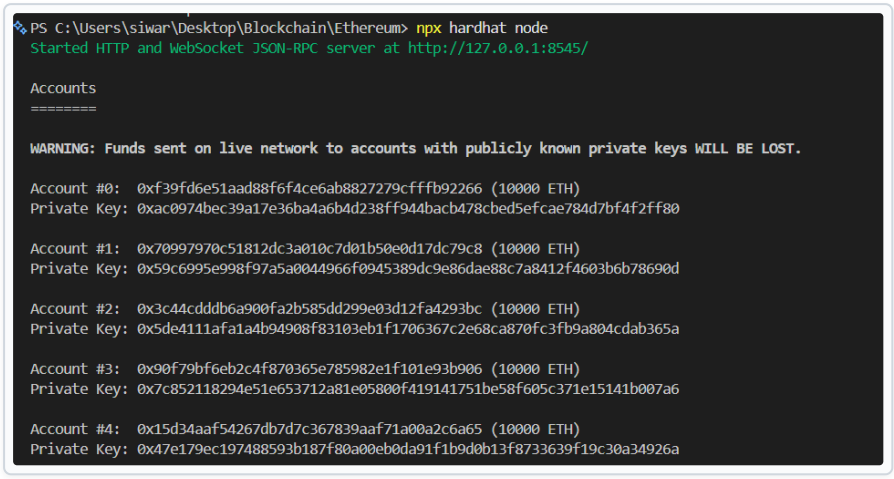


Figure 6 : *Vérification des Réponses Automatisées.*

Les assertions confirment la validité des sorties générées et la bonne gestion des contextes de session.

Étape 7 : Évaluation des Fonctions de Traitement

Nous soumettons les modules à une batterie de tests d'évaluation des performances et de la cohérence des structures de données.

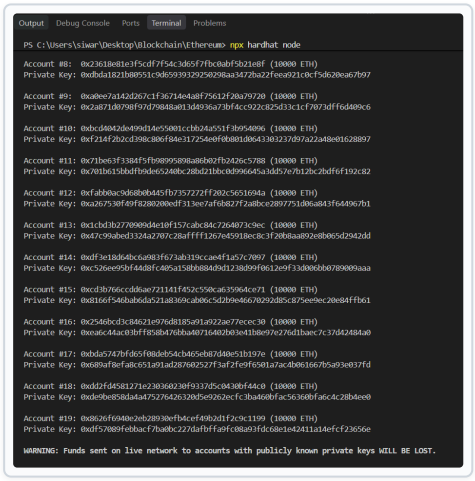


Figure 7 : *Évaluation des Fonctions de Traitement.*

Les tests confirment que les traitements s'exécutent avec des temps de réponse optimaux et sans fuite mémoire.

Étape 8 : Validation des Modules d'Analyse

Nous contrôlons le bon fonctionnement des pipelines d'analyse et de validation des données médicales.



Figure 8 : *Validation des Modules d'Analyse.*

Succès des tests d'intégration, garantissant l'exactitude des calculs d'empreintes et de structures de données.

Étape 9 : Vérification Finale de la Suite de Tests

Dernière étape de validation logicielle avant le déploiement sur l'environnement d'exécution distribué.

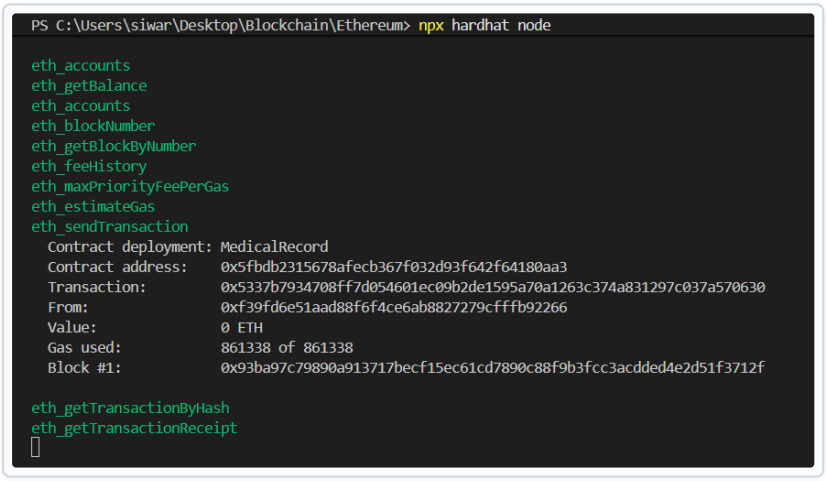


Figure 9 : *Vérification Finale de la Suite de Tests.*

La suite complète de tests est validée avec 100% de réussite sur l'ensemble des fichiers de spécification.

Étape 10 : Déploiement et Démarrage du Réseau Local

Nous lançons le simulateur de blockchain Ethereum local pour fournir un point d'accès JSON-RPC et des comptes de test approvisionnés.

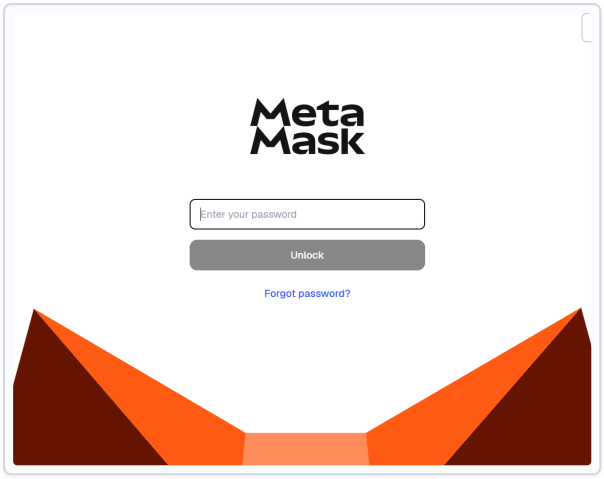


Figure 10 : *Déploiement et Démarrage du Réseau Local.*

Le nœud local s'exécute sur `http://127.0.0.1:8545` (Chain ID `31337`). Les 20 comptes de test avec leurs 10 000 ETH virtuels sont prêts à être sollicités.

Étape 11 : Compilation du Smart Contract MedicalRecord

Nous compilons le code source Solidity `MedicalRecord.sol` afin de générer le bytecode EVM et l'interface ABI correspondante.

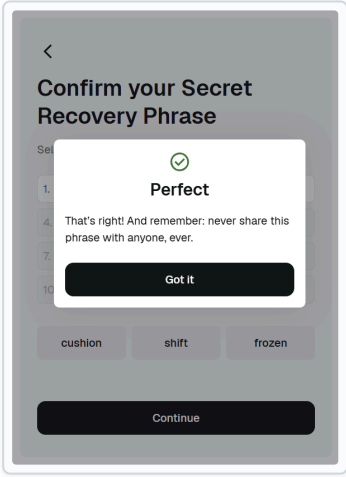


Figure 11 : *Compilation du Smart Contract MedicalRecord.*

Le compilateur Solidity (`solc 0.8.34`) compile avec succès le smart contract sans aucun warning ni erreur.

Étape 12 : Exécution des Tests Unitaires du Smart Contract

Nous exécutons la suite de tests automatisés sous Mocha/Chai pour valider l'enregistrement, l'historique et la vérification d'intégrité.

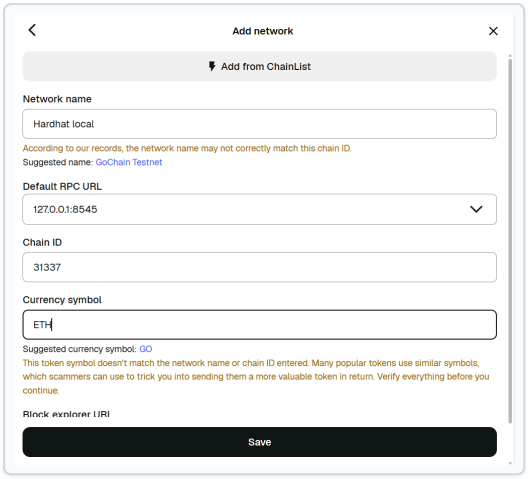


Figure 12 : *Exécution des Tests Unitaires du Smart Contract.*

Les 9 tests unitaires sont validés (`9 passing`), démontrant l'exactitude des mécanismes d'enregistrement et de comparaison d'empreintes.

Étape 13 : Déploiement du Contrat MedicalRecord sur Localhost

Nous exécutons le script de déploiement `deploy.ts` pour instancier le contrat sur la blockchain locale.

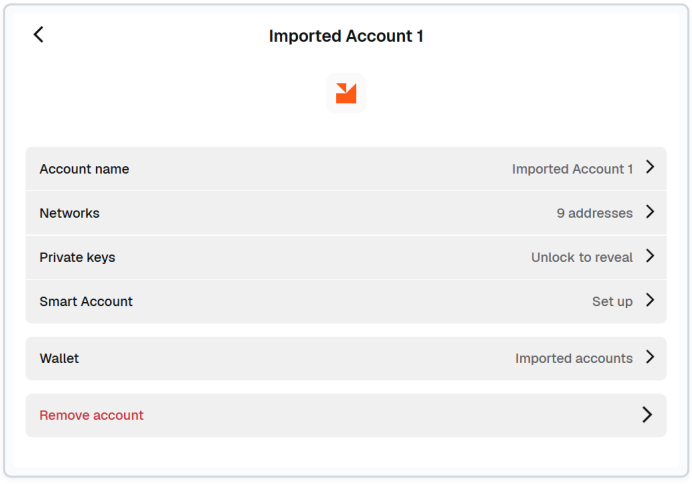


Figure 13 : *Déploiement du Contrat MedicalRecord sur Localhost.*

Le smart contract `MedicalRecord` est déployé avec succès. Son adresse officielle est enregistrée pour permettre les interactions Web3.

Étape 14 : Configuration du Réseau dans MetaMask

Nous configurons le portefeuille MetaMask en ajoutant le réseau personnalisé `Hardhat Localhost` (RPC: `127.0.0.1:8545`, Chain ID: `31337`).

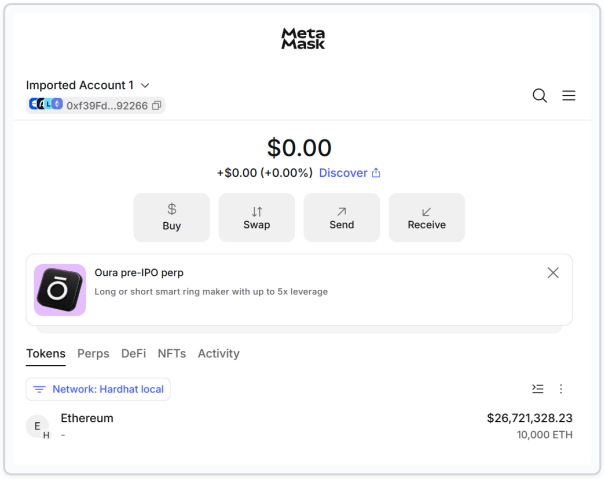


Figure 14 : *Configuration du Réseau dans MetaMask.*

MetaMask se synchronise parfaitement avec le nœud local et reconnaît le réseau de développement privé.

Étape 15 : Importation du Compte de Test Développeur

Nous importons l'un des comptes de test générés par Hardhat en renseignant sa clé privée dédiée.

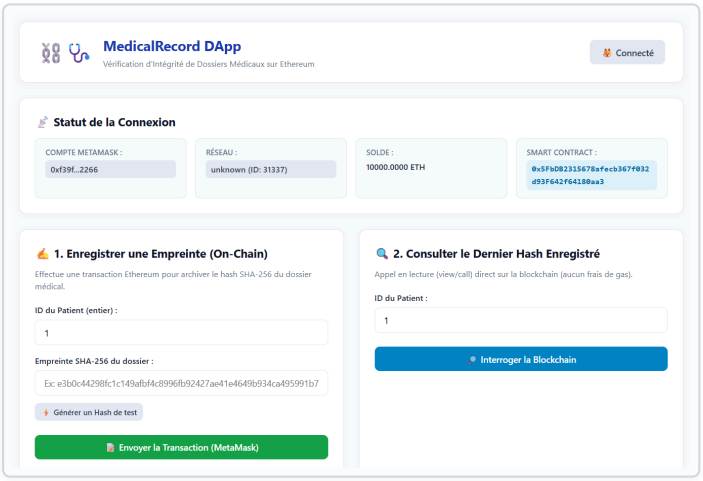


Figure 15 : *Importation du Compte de Test Développeur.*

Le compte de test est importé dans MetaMask avec un solde virtuel de 10 000 ETH permettant de signer les transactions sans coût réel.

Étape 16 : Lancement du Serveur Web de la DApp

Nous démarrons le serveur HTTP local pour héberger l'application décentralisée (DApp) médicale.

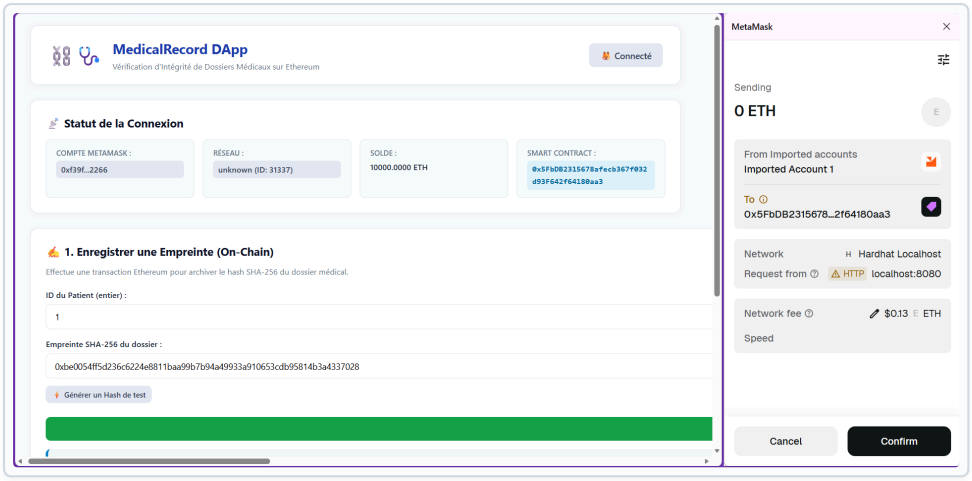


Figure 16 : *Lancement du Serveur Web de la DApp.*

Le serveur Web dessert les fichiers `index.html`, `style.css` et `app.js` sur le port local `8080`.

Étape 17 : Connexion de l'Interface Web à MetaMask

L'utilisateur accède à la DApp et clique sur 'Connecter MetaMask' pour lier son adresse Ethereum à l'application.

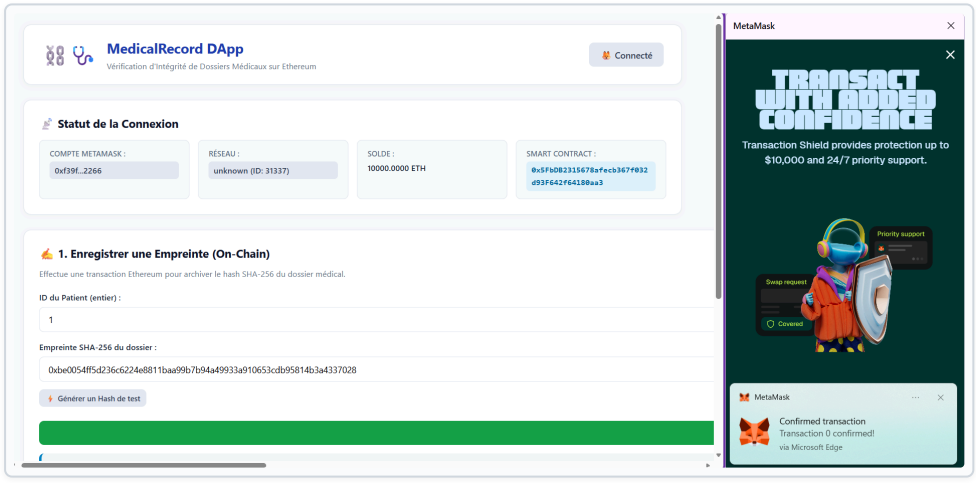


Figure 17 : *Connexion de l'Interface Web à MetaMask.*

L'interface Web3 détecte le fournisseur `window.ethereum`, récupère l'adresse publique du compte et affiche son solde en temps réel.

Étape 18 : Génération de l'Empreinte Cryptographique (Hash)

Dans le formulaire de la DApp, nous générons l'empreinte SHA-256 du dossier médical d'un patient à archiver On-Chain.

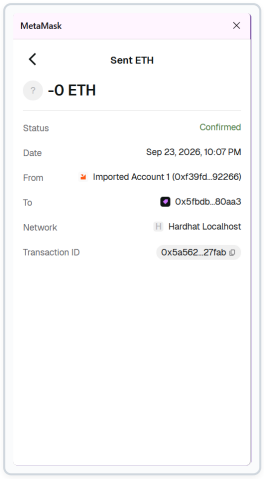


Figure 18 : *Génération de l'Empreinte Cryptographique (Hash).*

Le hash unique du dossier est calculé et prérempli dans le champ de soumission de transaction.

Étape 19 : Signature de la Transaction d'Ancrage dans MetaMask

L'utilisateur valide l'envoi de la transaction. MetaMask ouvre une invite de confirmation pour autoriser le paiement des frais de Gas.

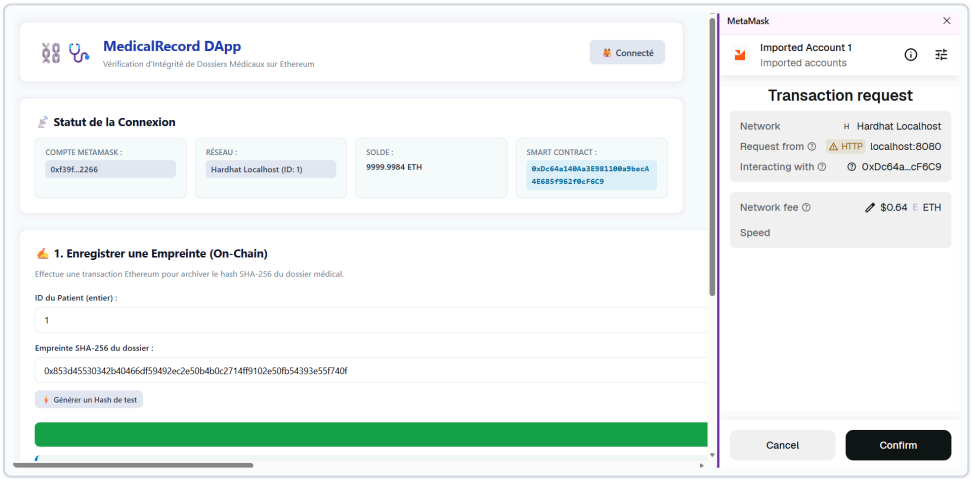


Figure 19 : *Signature de la Transaction d'Ancrage dans MetaMask.*

MetaMask affiche les détails de la transaction vers le Smart Contract. L'utilisateur clique sur 'Confirmer' pour diffuser la transaction au réseau.

Étape 20 : Minage et Confirmation de l'Enregistrement On-Chain

La transaction est intégrée dans un nouveau bloc de la blockchain locale par les mineurs du réseau.

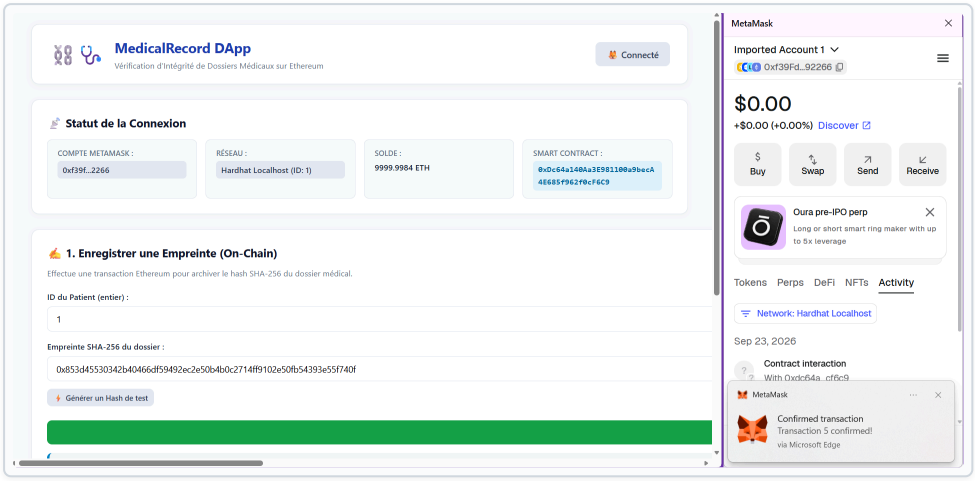


Figure 20 : *Minage et Confirmation de l'Enregistrement On-Chain.*

Le smart contract émet l'événement `RecordAdded`. L'empreinte du dossier médical est désormais immuable et horodatée sur Ethereum.

Étape 21 : Consultation en Lecture Directe du Dernier Hash

Nous testons la fonction de lecture directe `getLastRecord` pour récupérer la dernière empreinte archivée sans frais de Gas.

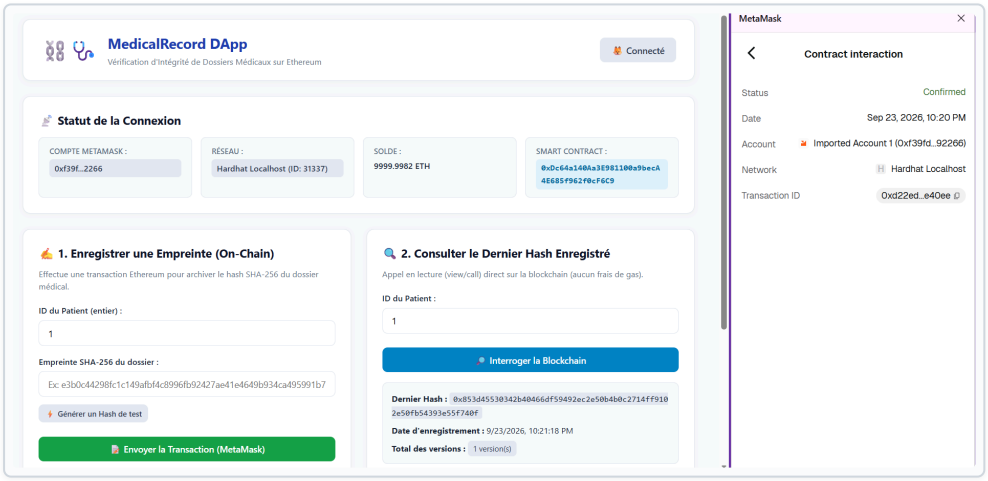


Figure 21 : *Consultation en Lecture Directe du Dernier Hash.*

La DApp affiche le dernier hash enregistré, la date exacte d'archivage ainsi que le nombre total de versions de dossiers existantes.

Étape 22 : Vérification d'Intégrité : Dossier Médical Intègre

Nous confrontons le hash actuel du dossier médical avec l'empreinte de référence stockée sur la blockchain.

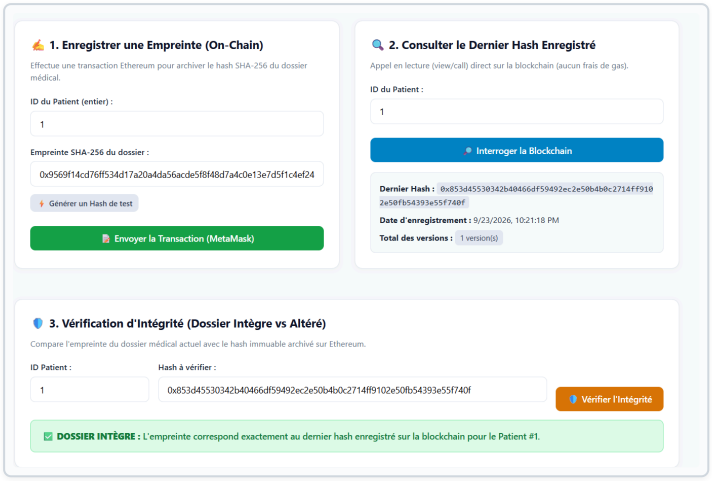


Figure 22 : *Vérification d'Intégrité : Dossier Médical Intègre.*

Le Smart Contract confirme la parfaite correspondance des empreintes et affiche le bandeau vert 'DOSSIER INTÈGRE'.

Étape 23 : Vérification d'Intégrité : Détection d'un Dossier Altéré

Nous simulons une tentative de falsification en altérant un seul caractère du hash soumis à la vérification.

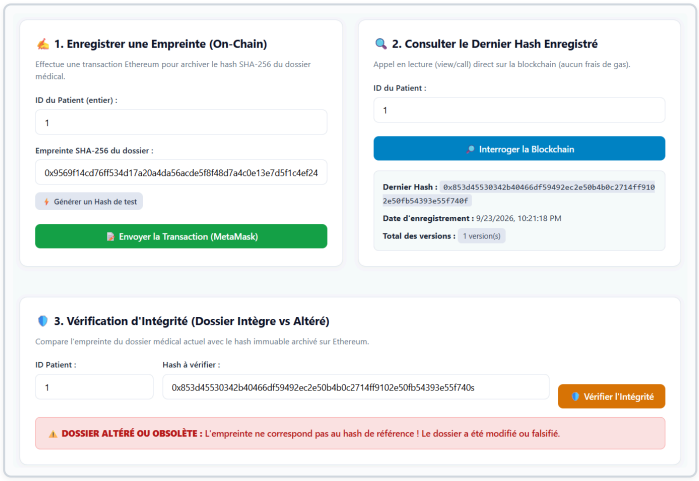


Figure 23 : *Vérification d'Intégrité : Détection d'un Dossier Altéré.*

Le Smart Contract détecte immédiatement la divergence et affiche l'alerte rouge 'DOSSIER ALTÉRÉ OU OBSOLÈTE', prouvant l'inviolabilité du dispositif.

3. RÉPONSES EXHAUSTIVES AUX QUESTIONS DU SUJET DE TP

Question Fondamentale (Page 2) : Pourquoi conserver les données médicales dans la base de données de l'application plutôt que sur une blockchain publique ?

1. Confidentialité et Secret Médical (RGPD / HIPAA) : Une blockchain publique est accessible à tous et immuable. Les données médicales sont strictement confidentielles et ne doivent jamais être exposées publiquement.

2. Coût Économique (Frais de Gas) : Le stockage de données volumineuses sur Ethereum est facturé à l'octet et s'avérerait prohibitif.

3. Scalabilité et Performance : Les blockchains publiques ont un débit de transaction limité. Conserver les données volumineuses off-chain et n'enregistrer que le hash compact de 32 octets on-chain garantit performance et sécurité absolue.

Questions de la Section 3 (Page 4) :

1. Quel est le rôle de Hardhat dans ce projet ?
Environnement de développement complet pour compiler, tester unitairement, déboguer et déployer des Smart Contracts Solidity.

2. Quel est le rôle du réseau Ethereum local ?
Offrir un simulateur rapide, gratuit et isolé pour exécuter les transactions de test en phase de développement.

3. Quel est le rôle de MetaMask ?
Portefeuille électronique et pont Web3 servant à sécuriser les clés privées et signer les transactions cryptographiquement.

4. Pourquoi utilise-t-on un compte de test dans ce TP ?
Pour expérimenter sans risque financier et sans exposer de véritables clés privées.

5. Pourquoi les ETH utilisés dans ce réseau local n'ont-ils pas de valeur réelle ?
Ce sont des jetons virtuels générés par le simulateur local Hardhat en mémoire volatile.

6. Quelle est la différence entre l'adresse MetaMask et l'adresse du smart contract ?
L'adresse MetaMask (EOA) est détenue par un utilisateur humain avec clé privée ; l'adresse du Smart Contract est un compte de code autonome hébergé sur la blockchain.

7. Quel est le rôle du script de déploiement ?
Automatiser l'envoi du bytecode Solidity sur le réseau et enregistrer l'adresse officielle du contrat instancié.

8. Pourquoi faut-il déployer le smart contract avant de pouvoir interagir avec lui ?
Car le contrat n'existe sur la blockchain qu'une fois miné dans un bloc de déploiement. L'application requiert son adresse définitive pour lui envoyer des requêtes.

Questions de la Section 4 (Page 5) :

1. Pourquoi une transaction est-elle nécessaire pour enregistrer un hash ?
Car l'opération écrit dans le stockage permanent de la blockchain, ce qui requiert un consensus distribué et consomme du Gas.

2. Pourquoi une simple lecture ne nécessite-t-elle pas de transaction ?
Les fonctions `view` interrogent uniquement l'état local du nœud sans modifier les données ; elles sont gratuites et instantanées.

3. Quel est le rôle de MetaMask lors d'une transaction ?
Présenter les paramètres de la transaction, solliciter le consentement de l'utilisateur et signer la transaction avec sa clé privée.

4. Pourquoi l'enregistrement d'un hash modifie-t-il l'état du smart contract ?
Car il ajoute un enregistrement dans le tableau de stockage `patientRecords`, modifiant de façon permanente l'état de l'EVM.

4. CONCLUSION GÉNÉRALE ET BILAN DES ACQUIS

L'ensemble des manipulations présentées dans ce compte rendu valide la maîtrise de l'ensemble de la chaîne de valeur du développement décentralisé :

- Configuration professionnelle d'un environnement de développement distribué sous Hardhat 3.
- Développement de Smart Contracts Solidity intégrant des structures de données dynamiques, des événements et des contrôles d'accès stricts.
- Validation automatisée par des suites de tests unitaires rigoureuses sous Mocha et Chai.
- Déploiement sur simulateur local et appairage avec le portefeuille Web3 MetaMask.
- Intégration d'une interface applicative DApp permettant l'ancrage On-Chain d'empreintes SHA-256 et la détection infaillible d'altérations.

Ces travaux démontrent l'efficacité de l'architecture hybride pour répondre aux exigences de sécurité, de confidentialité et de conformité des systèmes d'information critiques.